PREMIUM DEVELOPER EDITION

INFOR LN · DATABASE PROGRAMMING

Understanding *Embedded SQL*

in Infor LN — A Practical Guide

A comprehensive field guide to combining database operations with business logic in Infor LN. Master SELECT blocks, data operations, transaction management, and performance patterns trusted by enterprise developers.

Enterprise Developer Series
Infor LN · Baan · 4GL

FIRST EDITION · 2025

PREVIEW
FullOnBaan

CHAPTER ONE

The SELECT Block Structure

How Embedded SQL wraps every outcome of a query

> *Embedded SQL in Infor LN doesn't just run a query — it hands you the result in a structured envelope, with dedicated handlers for every possible state your data might be in.*

Unlike raw SQL executed against a database client, Embedded SQL lives inside your application code. Every SELECT statement is surrounded by a set of control blocks that define exactly what happens when records are found, when the dataset is exhausted, when it comes back empty, and when something goes wrong.

Basic SELECT Syntax

basic_select.4gl — INFOR LN · 4GL

```
select tablename.*
from  tablename
where (conditions)
order by ...
group by ...

selectdo
  -- Business logic for each filtered record
  -- Executes for every record except the last
selecteos   -- "End Of Select" — fires for the last record

selecterror
  message("Custom Error Message")
selectempty
  message("Record not found")
endselect
```

Control Block Reference

SELECTDO

Executes once per returned record. Place all per-record business logic here. Runs for every record *except* the final one.

SELECTEOS

Fires exactly once — for the very last record in the result set. Ideal for totals, summary calculations, or final output.

SELECTEMPTY

Executes when the query returns zero records. Never leave this unhandled — always provide meaningful feedback or fallback logic.

SELECTERROR

Catches runtime errors during the SELECT operation. Use `message()` to surface human-readable error messages.

Think in States, Not Just Queries

Embedded SQL forces you to handle every outcome explicitly. This discipline — defining what happens when data is found, not found, ends, or errors — is what makes Infor LN applications resilient and production-ready.

PREVIEW
FullOnBaan

CHAPTER TWO

Data Operations

Insert · Update · Delete — the safe & consistent pattern

> *All three data operations in Embedded SQL share a single guiding principle: always query before you act. The SELECT block tells you what exists — then you decide what to do.*

INSERT Check for existence first · Insert only if record is absent

insert_operation.4gl

```
01  db.retry.point()
02  select tablename.*
03  from  tablename
04  where tablename._index1 = :value1
05  selectdo
06    message("Record already exists")
07  selectempty                          -- Safe to insert
08
09    tablename.field1 = value1
10    tablename.field2 = value2
11    db.insert(tablename, db.retry)
      commit.transaction()
    endselect
```

UPDATE Lock row with FOR UPDATE · Modify only confirmed records

update_operation.4gl

```
01  db.retry.point()
02  select tablename.*
03  from  tablename for update    -- Locks the row
04
05  where tablename._index1 = :value1
06  selectdo
07    tablename.field2 = value2
08    db.update(tablename, db.retry)
09    commit.transaction()
10  selectempty
```

PREVIEW FullOnBaan

```
    message("Record not found")
  endselect
```

DELETE Same FOR UPDATE pattern · Remove only what's confirmed

delete_operation.4gl

```
db.retry.point()
select tablename.*
from  tablename for update
where tablename._index1 = :value1
selectdo
  db.delete(tablename, db.retry)
  commit.transaction()
selectempty
  message("Record not found")
endselect
```


CHAPTER THREE

Commit Transaction Placement

Critical decisions for performance & data integrity

> *Where you place `commit.transaction()` is one of the most consequential architectural decisions in any Infor LN program. Get it wrong and you'll face locking issues, data loss, or crippling performance.*

The four placement options each represent a different trade-off between safety and throughput. Understanding them — and choosing deliberately — separates experienced LN developers from the rest.

commit_placement.4gl ALL 4 OPTIONS

```
select * from tablename for update where condition
selectdo
  counter = counter + 1
  db.update(tablename, db.retry)
  commit.transaction()    -- Option 1 · after every record
```

PREVIEW
FullonBaan